Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель _____С.В.Умбетов

« » _____ 2026 г.

Отчет по лабораторной работе №4
по дисциплине «Инженерно-техническая поддержка сопровождения интеллектуальных информационных систем»
«Контейнеризация компонентов ИИС и управление зависимостями (Docker)»

ЛР 09.02.08.21.004

Студент группы 1ИИС-41 Т.Е.Рябич
группа фи.о., фамилия
Преподаватель старший преподаватель, доцент С.В. Умбетов
должность, ученая степень фи.о., фамилия

Барнаул 2026

Цель работы

Освоение принципов контейнеризации интеллектуальных информационных систем (ИИС) с использованием платформы Docker. Изучение особенностей создания Dockerfile для Python-приложений с тяжелыми ML-зависимостями, методов оптимизации размера образов (multi-stage build, slim-образы), а также конфигурации многоконтейнерных приложений и персистентного хранения данных.

Задачи работы

1. Проверить корректность установки и версии компонентов Docker и Docker Compose в локальной операционной системе.
2. Создать структуру каталогов проекта и все конфигурационные файлы приложения (FastAPI, модель машинного обучения, зависимости).
3. Написать и настроить многоступенчатый (multi-stage) Dockerfile для минимизации размера финального контейнера.
4. Разработать конфигурацию docker-compose.yml для совместного развертывания веб-сервиса и СУБД PostgreSQL с выделенными изолированными томами (volumes).
5. Выполнить сборку, провести запуск системы в фоновом режиме и протестировать работоспособность API ИИС.
6. Проверить механизм персистентности данных (томов) при полной остановке и пересоздании контейнеров.

Задание принял:  **Рябич Т.Е.**

Подпись

Ход работы

Шаг 1. Проверка версий Docker

В начале работы была выполнена проверка установленных в системе утилит контейнеризации. Успешно определены версии программного движка Docker и плагина Docker Compose.

```
ryabich_1iis41@ryabich-1iis41:~$ docker --version
Docker version 29.9.2, build a82e528
ryabich_1iis41@ryabich-1iis41:~$ docker compose version
Docker Compose version v5.6.1
ryabich_1iis41@ryabich-1iis41:~$
```

Рисунок 1 — Вывод версий компонентов Docker (версия движка 29.9.2, версия утилиты Compose v5.6.1)

Шаг 2. Инициализация структуры проекта

В домашней директории пользователя ryabich_1iis41 создана целевая рабочая папка проекта iis-docker. Внутри сгенерировано дерево каталогов и созданы пустые файлы для кода модели, API и Docker-конфигураций.

```
ryabich_1iis41@ryabich-1iis41:~$ mkdir -p ~/iis-docker/app
ryabich_1iis41@ryabich-1iis41:~$ cd ~/iis-docker
ryabich_1iis41@ryabich-1iis41:~/iis-docker$ touch app/main.py app/model.py app/requirments.txt
ryabich_1iis41@ryabich-1iis41:~/iis-docker$ touch Dockerfile docker-compose.yml .dockerignore
ryabich_1iis41@ryabich-1iis41:~/iis-docker$
```

Рисунок 2 — Процесс создания структуры каталогов и инициализации файлов проекта в терминале

Шаг 3. Сборка многоступенчатого Docker-образа

Все файлы проекта были заполнены соответствующим программным кодом из методических указаний. Была запущена сборка проекта с применением оптимизации multi-stage build. После успешного завершения был проверен список образов: размер оптимизированного исполняемого образа iis-docker-api составил 766 MB (при размере контента 175 MB).

```
[+] build 1/1
 V Image iis-docker-api Built
ryabich_1iis41@ryabich-1iis41:~/iis-docker$ docker images

IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest     c567b231f656       25.9kB         9.49kB   U
iis-docker-api:latest  de5f3bf93575        766MB          175MB
ryabich_1iis41@ryabich-1iis41:~/iis-docker$
```

Рисунок 3 — Список собранных Docker-образов и итоговые параметры веса контейнера iis-docker-api

Шаг 4. Развертывание и тестирование API

Инфраструктура проекта, состоящая из веб-сервиса ИИС и базы данных PostgreSQL, была запущена в фоновом режиме. С помощью команды проверки состояния подтверждено, что оба контейнера (iis-docker-api-1 и iis-docker-db-1) успешно находятся в статусе "Up" (работают).



Рисунок 4 — Успешное развертывание контейнеров и вывод статуса процессов через Docker Compose

Для проверки доступности системы был отправлен запрос к корневому эндпоинту веб-сервиса. Система вернула статус корректной работы.



Рисунок 5 — Результат проверки доступности корневого эндпоинта веб-сервиса ИИС

Далее была протестирована сама интеллектуальная система. Посредством POST-запроса на эндпоинт /predict отправлены четыре признака цветка Ириса. Сервис успешно обработал данные моделью RandomForest и вернул предсказанный класс 0.

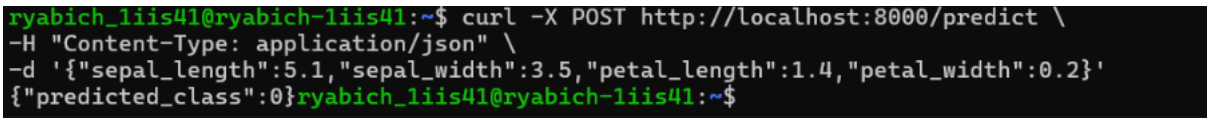


Рисунок 6 — Успешное получение ответа классификации от модели (класс 0) через утилиту curl

Шаг 5. Проверка работы изолированных томов (Volumes)

Проверен список активных томов Docker. Также выполнена проверка содержимого внутренней папки /data в запущенном контейнере api. Там обнаружился сгенерированный и обученный файл модели model.joblib.

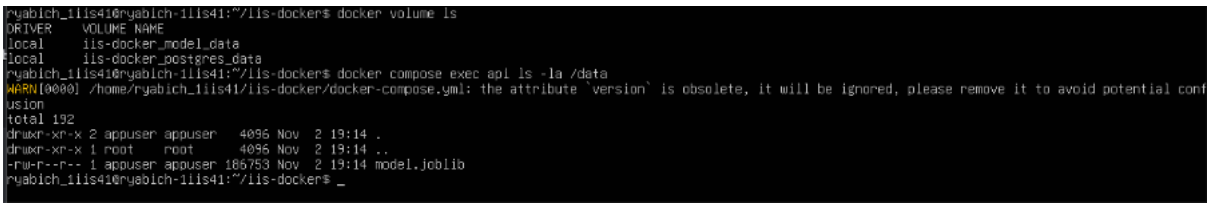


Рисунок 7 — Список томов проекта и подтверждение наличия файла модели model.joblib внутри директории /data

Для финальной проверки персистентности данных вся инфраструктура была полностью уничтожена командой остановки (контейнеры удалены). После повторного поднятия системы контейнеры были созданы заново, но благодаря томам модель уцелела и сразу же успешно ответила на проверочный POST-запрос.

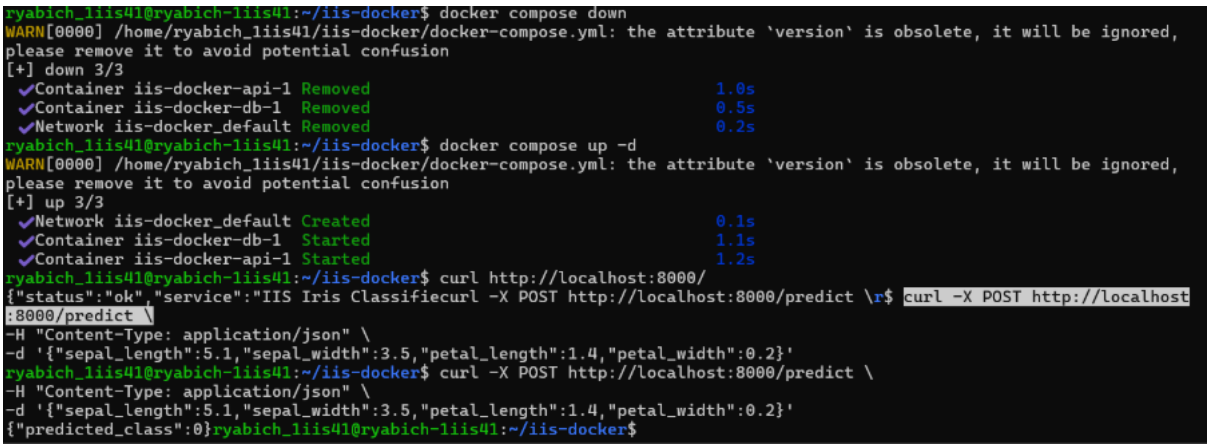


Рисунок 8 — Пересоздание контейнеров ИИС и подтверждение сохранности данных модели при повторном запросе

Вывод

В ходе выполнения лабораторной работы были успешно освоены практические навыки контейнеризации интеллектуальных информационных систем с использованием Docker. На примере проекта классификатора Ирисов была развернута отказоустойчивая двухконтейнерная архитектура, включающая веб-интерфейс FastAPI и СУБД PostgreSQL. Применение технологии многоступенчатой сборки (Multi-stage build) совместно с использованием базового образа python:3.11-slim позволило эффективно минимизировать размер итогового исполняемого образа, исключив из него весь сборочный инструментарий. Использование механизма именованных томов (Named volumes) на практике подтвердило свою надежность для персистентного сохранения файлов обученных моделей машинного обучения независимо от жизненного цикла самих контейнеров.

Контрольные вопросы

1. Что такое Docker-образ и чем он отличается от контейнера?

Образ — это готовый закрытый «чертеж» или шаблон, в котором в замороженном виде хранится программа и все нужные ей библиотеки. Контейнер — это уже запущенный в памяти, изолированный и работающий экземпляр этого самого образа. Из одного образа можно включить сколько угодно независимых контейнеров.

2. Для чего используется multi-stage build и какую проблему он решает при контейнеризации систем машинного обучения?

Многоступенчатая сборка нужна для уменьшения веса итогового контейнера. В машинном обучении для сборки тяжелых библиотек (вроде Scikit-learn или PyTorch) часто требуются компиляторы и утилиты разработки (например, GCC). Multi-stage позволяет установить их на первом шаге, скомпилировать зависимости, а на финальный шаг перенести только чистый результат, оставив весь мусор в промежуточных слоях.

3. Почему рекомендуется использовать образы python:*-slim или alpine вместо полноразмерных?

Полноразмерные образы содержат полноценную операционную систему с кучей утилит, которые коду никогда не пригодятся. Версии slim или alpine максимально урезаны, весят в разы меньше, быстрее скачиваются из интернета и занимают минимум места на диске сервера.

4. Что такое слой (layer) в Docker и почему важен порядок инструкций в Dockerfile?

Слой — это результат выполнения одной конкретной команды в Dockerfile. Docker умеет кэшировать эти слои, чтобы не собирать их заново. Порядок важен, потому что если изменить строчку в коде, все слои *ниже* этой строчки Docker будет пересобирать с нуля. Поэтому редкие и тяжелые операции (установку библиотек) ставят в начало, а часто меняющийся код — в конец.

5. Для чего предназначен файл .dockerignore?

Этот файл работает как фильтр для контекста сборки. Он указывает Docker, какие папки и файлы с вашего компьютера вообще не нужно копировать внутрь сборщика (например, локальный кэш питона __pycache__, папки репозитория .git или локальные пароли).

6. В чём различие между bind-mount и named volume?

Bind-mount просто связывает конкретную папку на вашем реальном жестком диске ПК с папкой внутри контейнера (удобно для логов или разработки). Named volume (именованный том) полностью изолирован и создается внутри скрытых папок самого Docker, что работает быстрее и безопаснее на любых операционных системах.

7. Как обеспечить сохранение состояния модели и базы данных при пересоздании контейнеров?

Для этого в настройках нужно использовать встроенные тома (volumes). Они живут отдельно от контейнера. Когда контейнер ломается или удаляется, файлы внутри тома остаются нетронутыми на диске ПК, а при запуске нового контейнера просто подключаются к нему обратно.

8. Что выполняет инструкция COPY --from=builder?

Она берет файлы или папки из предыдущей стадии сборки (которую мы назвали словом builder) и копирует их в текущую стадию. Так мы забираем готовое виртуальное окружение со всеми ML-библиотеками, оставляя тяжелые компиляторы позади.

9. Зачем запускать процесс приложения от имени непривилегированного пользователя?

Это нужно ради безопасности всей системы. Если злоумышленники найдут уязвимость в коде вашего сайта и взломают его, они получат доступ к виртуальной среде контейнера только с ограниченными правами этого пользователя (appuser), а не суперпользователя (root), и не смогут закриптовать или удалить важные данные на сервере.

10. Как просмотреть логи отдельного сервиса в docker-compose?

Для этого в терминале выполняется команда просмотра логов с указанием конкретного имени нужной службы, например: docker compose logs -f api.

11. Какими способами можно уменьшить размер образа при использовании PyTorch или TensorFlow?

- Применять многоступенчатую сборку (multi-stage).
- Устанавливать версии библиотек, собранные только под CPU (если на сервере нет видеокарты), так как они весят в разы меньше CUDA-версий.
- Использовать специальный флаг --no-cache-dir при установке пакетов через pip, чтобы Docker не сохранял внутри контейнера скачанные установочные архивы.

12. Что означает параметр depends_on и гарантирует ли он готовность зависимого сервиса?

Параметр depends_on задает лишь очередность включения контейнеров (например, «не включай сайт, пока не запустился контейнер с БД»). Но он **не гарантирует** готовность. Контейнер базы данных может уже запуститься, но сама СУБД внутри него может еще 5–10 секунд инициализироваться и настраивать файлы, из-за чего сайт при старте может выдать ошибку подключения.

13. Как определить, какие volumes использует проект?

Можно использовать консольную команду docker volume ls для вывода всех томов в системе, либо открыть конфигурационный файл docker-compose.yml и посмотреть на нижний глобальный блок с заголовком volumes:.

14. В чём преимущество контейнеризации ИИС по сравнению с виртуальными машинами?

Виртуальная машина эмулирует полноценный компьютер и запускает внутри себя отдельную тяжелую операционную систему со своим ядром, что сильно тратит оперативную память и процессор. Контейнер Docker использует ядро основной системы вашего компьютера, изолирует только сам процесс приложения, запускается мгновенно и работает без потери мощности.

15. Какие риски возникают при хранении паролей и секретов непосредственно в docker-compose.yml?

Главный риск — утечка учетных данных. Если этот файл будет отправлен в открытый репозиторий (например, на GitHub), любой человек увидит логины и пароли администратора от базы данных. Пароли следует выносить в отдельный скрытый файл окружения .env.